Information Sciences 575 (2021) 265–288

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Systematic categorization and evaluation of CbO-based algorithms in FCA

Jan Konecny *, Petr Krajča

Dept. Computer Science, Palacký University Olomouc, 17. listopadu 12, Olomouc CZ-77146, Czech Republic

ARTICLE INFO

Article history:
Received 29 July 2020
Received in revised form 11 March 2021
Accepted 9 June 2021
Available online 12 June 2021

Keywords:
Formal concept analysis
Close-by-One
Algorithm
Performance
Comparison

ABSTRACT

Algorithms based on Close-by-One (CbO) are polynomial delay algorithms used for the enumeration of closed sets, particularly formal concepts in Formal Concept Analysis. We describe and categorize their distinctive features. We experimentally evaluate the influence of the features on the computation time. We show that via the study of individual features, we can design new and more efficient algorithms.

© 2021 Published by Elsevier Inc.

1. Introduction

Closed sets are of crucial importance in multiple areas of computer sciences. They concisely represent frequent itemsets in association analysis [32,36], they serve as optimal and universal factors in boolean matrix factorization [14], they are used as the basic building blocks for hypothesis generation in logical data analysis [5,3], and they are the primary interest in Formal Concept Analysis (FCA) [21,16].

Many algorithms for closed sets enumeration exist [16,28,4]. In FCA, In-Close family [6–10] and FCbO [31] are considered to be the state-of-the-art algorithms. The basis for all these algorithms is the algorithm Close-by-One (CbO), introduced by Kuznetsov [27]. Each of the algorithms uses a specific combination of features which optimizes and speeds up the computation. We identify the following three features:

- sweep direction – the order in which an algorithm visits the nodes of the search tree;
- partial closures – feature which allows for computing only a necessary part of a closed set and finishing the computation when its uniqueness is assured;
- pruning – the way in which the search tree is reduced by avoiding some redundant computations.

We describe the features and experimentally evaluate all their plausible combinations in search of the 'best-of-breed' algorithm.

In the paper, we assume only the basic setting of FCA. However, all the algorithms presented in this paper can be straightforwardly elevated into a more general setting of FCA, as long as it has distinguishable attributes and its concept-forming

* Corresponding author.
E-mail addresses: jan.konecny@upol.cz (J. Konecny), petr.krajca@upol.cz (P. Krajča).

https://doi.org/10.1016/j.ins.2021.06.024
0020-0255/© 2021 Published by Elsevier Inc.

operators form a Galois connection. Among such settings are, for example, those which handle **L**-fuzzy attributes [13,15], three-way attributes [33,37,34,38], attribute values from multi-adjoint structures [30,17], real-valued and interval-valued setting [29], linguistic setting [39], multivalued data [1,2] and positive and negative attributes [11,12,24].

We consider the following to be the main contribution of the paper:

- A systematic review of CbO-based algorithms in FCA and standalone descriptions of their speed-up features.
- An evaluation of all combinations of speed-up features. Closed set mining is useful in many areas and efficient approaches are desired.
- The observation that the pruning techniques (described in Section 6) are compatible with a left depth-first sweep direction. This is important for
  - new algorithms – there are five new CbO-based algorithms using this fact described and evaluated in this paper.
  - a use of CbO-based algorithms in frameworks where a combined sweep (described in Section 4) would be cumbersome. An example of such a framework is Map-Reduce [18]. In [25], we successfully applied this and provided a CbO with pruning for Apache Spark and/or similar programming models [35].

The rest of this paper is structured as follows: first, we recall the basic notions of FCA (Section 2) along with the CbO algorithm (Section 3). Then, we describe the speed-up features, namely, sweep direction (Section 4), partial closures (Section 5), and pruning (Section 6). With the related features, we also describe the algorithms which utilize them. We comment on the mutual compatibility of the features. Then, we provide a system covering CbO-based algorithms used in FCA and many new algorithms (Section 7). We perform a thorough experimental evaluation of all the algorithms in the system (Section 8). Finally, we summarize our conclusions and present directions for our future research.

The paper has two appendices:

A shows a standalone description of three CbO-based algorithms which turned out to be the most efficient in the experimental evaluation.

B is a userguide to an interactive HTML page with a complete set of charts depicting a performance comparison of CbO-based algorithms.

2. Formal concept analysis

An input to FCA is a triplet $\langle X, Y, I\rangle$, called a *formal context*, where X, Y are non-empty sets of objects and attributes respectively, and I is a binary relation between X and Y. The presence of an object-attribute pair $\langle x, y\rangle$ in the relation I means that the object x has the attribute y.

Finite contexts are usually depicted as tables, in which rows represent objects in X, columns represent attributes in Y, $\times$s in its entries mean that the corresponding object-attribute pair is in I; see Fig. 3 for an example.

The formal context $\langle X, Y, I\rangle$ induces so-called *concept-forming operators*:

$^{\uparrow} : \mathbf{2}^X \to \mathbf{2}^Y$ assigns to a set A of objects the set $A^{\uparrow}$ of all attributes shared by all the objects in A.
$^{\downarrow} : \mathbf{2}^Y \to \mathbf{2}^X$ assigns to a set B of attributes the set $B^{\downarrow}$ of all objects which share all the attributes in B.

Formally, for all $A \subseteq X, B \subseteq Y$ we have

$$A^{\uparrow} = \{y \in Y \mid \forall x \in A : \langle x, y\rangle \in I\},$$
$$B^{\downarrow} = \{x \in X \mid \forall y \in B : \langle x, y\rangle \in I\}.$$

For singletons, we use shortcut notation and write $i^{\downarrow}$ instead of $\{i\}^{\downarrow}$.

The composition $((\cdot)^{\downarrow})^{\uparrow}$ of the concept-forming operators, shortened as $(\cdot)^{\downarrow\uparrow}$, is a closure operator on Y, i.e. it satisfies the following conditions:

$$B \subseteq B^{\downarrow\uparrow} \tag{1}$$
$$B_1 \subseteq B_2 \text{ implies } B_1^{\downarrow\uparrow} \subseteq B_2^{\downarrow\uparrow} \tag{2}$$
$$B^{\downarrow\uparrow\downarrow\uparrow} = B^{\downarrow\uparrow} \tag{3}$$

for all $B, B_1, B_2 \subseteq Y$.

Analogously, the composition $(\cdot)^{\uparrow\downarrow}$ is a closure operator on X.

Fixed points of the concept-forming operators, i.e. pairs $\langle A, B\rangle \in \mathbf{2}^X \times \mathbf{2}^Y$ satisfying $A^{\uparrow} = B$ and $B^{\downarrow} = A$, are called *formal concepts*. The sets A and B in a formal concept $\langle A, B\rangle$ are called the *extent* and the *intent*, respectively.

We can characterize intents as sets $B \subseteq Y$ satisfying $B = B^{\downarrow\uparrow}$, that is fixed points of the closure operator $(\cdot)^{\downarrow\uparrow}$. Furthermore, $B^{\downarrow\uparrow}$ is the smallest intent containing B.

Intents are exactly closed itemsets in association analysis [32,36]. Spanned patterns in Logical Analysis of Data are those patterns which are intents [5,22]. Optimal and universal factors in boolean matrix factorization are formal concepts [14]. Maximal bicliques in bipartite graphs are formal concepts.

Algorithms described in Sections 1–6 enumerate the set $\mathscr{B}(X,Y,I)$ of all formal concepts in $\langle X,Y,I\rangle$. Note that it is enough to enumerate the set $\mathrm{Int}(X,Y,I)$ of all intents, since we can simply derive $\mathscr{B}(X,Y,I)$ by[1]

$$\mathscr{B}(X,Y,I) = \{\langle B^{\downarrow}, B\rangle : B \in \mathrm{Int}(X,Y,I)\}. \tag{4}$$

While the algorithms indeed output only intents, they can be easily altered to output entire formal concepts. We do not need to add a post-processing step corresponding to (4) since the extents are computed as intermediate results.

3. Close-by-One

We assume input formal context $\langle X,Y,I\rangle$ where $Y = \{1,2,\ldots,n\}$. Whenever we write about lower attributes or higher attributes, we refer to the natural ordering of the numbers in Y.

3.1. Naïve Approach

We start with a naïve algorithm for enumerating all subsets of Y (shown in Algorithm 1).

Algorithm 1: Naïve algorithm for enumerating all intents of $\langle X,Y,I\rangle$ with concept-forming operators $\uparrow,\downarrow$

```
def GenerateFrom(B, y):
    arguments: B – set of attributes
               y – last added attribute

1   if B = B↓↑ then print(B)
2   for i ← y + 1 to n do
3       D ← B ∪ {i}
4       GenerateFrom(D, i)

GenerateFrom(∅, 0)
```

The algorithm is given by a recursive procedure `GenerateFrom` which accepts two arguments:

- B – set of attributes, from which new sets are generated.
- y – auxiliary argument to store the last attribute added to B.

The procedure first prints out the input set B if it is an intent (line 1). Then, for each attribute i higher than the highest attribute y in B:

- a new set is generated by adding the attribute i into the set B (line 3);
- the procedure recursively calls itself to process the new set (line 4).

The procedure is initially called with an empty set and zero as its arguments.

The algorithm represents a left depth-first sweep through the tree of all subsets (see Fig. 1 (top)) and filtering those which satisfy $B = B^{\downarrow\uparrow}$.

This approach is quite inefficient as the number of intents is typically much smaller than the number of all subsets.

CbO exploits closure properties of the composition $(\cdot)^{\downarrow\uparrow}$. Recall that $(B \cup \{i\})^{\downarrow\uparrow}$ is the smallest intent containing $B \cup \{i\}$.

In the tree of all subsets (Fig. 1 (top)), each node is a superset of its predecessors. We can use the closure operator $(\cdot)^{\downarrow\uparrow}$ to skip non-intents; in other words, we make jumps in the tree to intents only.

Instead of simply adding an element to generate a new subset

$$D \leftarrow B \cup \{i\},$$

CbO adds the element and then applies the closure operator

$$D \leftarrow (B \cup \{i\})^{\downarrow\uparrow}. \tag{5}$$

[1] Equivalently, this holds true for extents.

We need to distinguish the two outcomes of (5). Either

- the resulting intent contains some attributes lower than i not included in B, i.e.

  $$D_i \neq B_i$$

  where $D_i = D \cap \{1, \ldots, i-1\}$, $B_i = B \cap \{1, \ldots, i-1\}$;
- or it does not, and we have

  $$D_i = B_i.$$

  The jumps with $D_i \neq B_i$ are not desirable because they land on intents which were already processed. As such jumps would lead to intents outputted multiple times, CbO does not perform them. The check of the condition $D_i = B_i$ is called the *canonicity test*.

For sake of efficiency, we can exploit properties of extents and split the operation of adding and closing (5) into two operations,

$$\begin{aligned} C &\leftarrow A \cap i^{\downarrow}, \\ D &\leftarrow C^{\uparrow}, \end{aligned} \tag{6}$$

where A is the extent corresponding to B. Notice that $(B \cup \{i\})^{\downarrow}$ is a more computationally intensive task than $A \cap i^{\downarrow}$.

The pseudocode of CbO is shown in Algorithm 2. We shall describe differences from the naïve approach. First, the input set B for the procedure `GenerateFrom` is already closed, therefore we do not need to check whether $B = B^{\downarrow\uparrow}$ as in the naïve approach, and we directly print out the pair $\langle A, B \rangle$ (line 1). The new set D is generated by adding i into B and subsequently closing using (6) (lines 4,5). The recursive call is made only if D satisfies the canonicity test (lines 6,7).

Algorithm 2: Close-by-One

```
   def GenerateFrom(A ,B, y):
       input : A – extent
               B – intent
               y – last added attribute
1      print(⟨A, B⟩)
2      for i ← y + 1 to n do
3          if i ∉ B then
4              C ← A ∩ i↓
5              D ← C↑
6              if D_i = B_i then
7                  GenerateFrom(C, D, i)
   GenerateFrom(X, X↑, 0)
```

Remark 1. [Time complexity] CbO has a polynomial delay in $\mathcal{O}(|X| \cdot |Y|^2)$. That means that the time between the start of the algorithm and output of the first concept, the time between between the output of two consequent concepts, and the time between outputting the last concept and the termination of the algorithm is in $\mathcal{O}(|X| \cdot |Y|^2)$ [23]. CbO's total time complexity is thusly in $\mathcal{O}(|X| \cdot |Y|^2 \cdot |\mathscr{B}(X, Y, I)|)$. This holds true for all CbO-based algorithms considered in this paper.

4. Sweep direction

Traditionally, CbO is described as performing the left depth-first sweep through the tree of all (closed) subsets. However, CbO does not require any particular order of direction of the traversal. We can also use right depth-first sweep, so-called combined sweep (see below) or any other systemic sweep. This also enables us to easily parallelize the algorithm [26].

We can alter the pseudo-code of CbO (Algorithm 2) to perform the right depth-first sweep (Fig. 1 (bottom)) by simply reverting the direction of the main cycle (line 2):

2 **for** $i \leftarrow n$ to $y + 1$ **do** ...

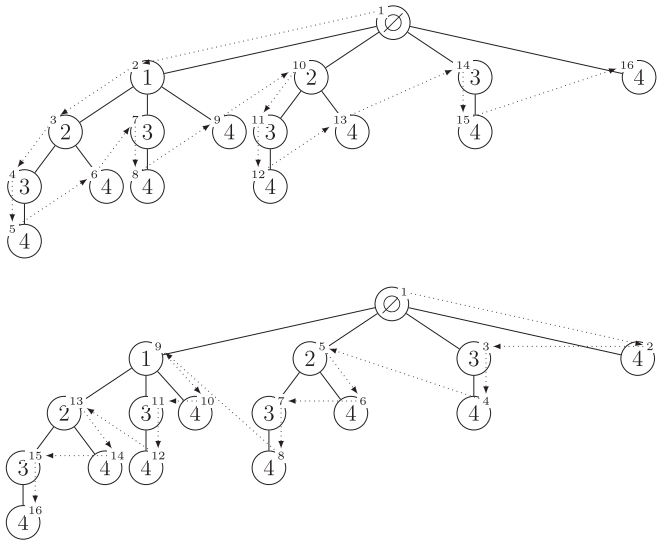


Fig. 1. Left depth-first sweep (top) and right depth-first sweep (bottom).

At first glance, a change of the sweep order does not seem to be a speed-up feature – it is usually introduced as a requirement for another feature. However, we will see later in experimental evaluation that the change itself, i.e. without the feature which requires it, can make a significant difference in the performance.

Remark 2. [Algorithm Next Closure] Ganter & Wille [21] describe the algorithm NextClosure. It utilizes a linear order, called lectic order, on 2^Y and defines an operator $(\cdot)^+ : 2^Y \to 2^Y$ as

$$(B)^+ = (B_i \cup \{i\})^{\downarrow\uparrow} \quad \text{where } i \text{ is the maximal attribute satisfying } (B)^+_i = B_i,$$

for all $B \in 2^Y$. For B being an intent, $(B)^+$ is proven to be the smallest intent which is lectically higher than B. Starting from $\varnothing^{\downarrow\uparrow}$, NextClosure iteratively applies the operator $(B)^+$ to generate all intents until it reaches $B = Y$. While it is not obvious from the description, the NextClosure performs the same traversal through the tree of all subsets as CbO with the right depth-first sweep. With a little simplification, the cut B_i can be seen as a return to a predecessor if i is lower than i from the previous iteration or descend to a successor if it is higher. The rest of the operator $(\cdot)^+$ is adding i and closing as in (5). The condition $(B)^+_i = B_i$ is the test of canonicity. The maximality of i matches with the right-first ordering of visited child nodes.

The algorithms FCbO and In-Close2 (and higher versions thereof) utilize a particular combination of the depth-first and the breadth-first search. Specifically, when processing a node, all its child nodes are visited first, then the algorithm processes the corresponding subtrees in the same manner (Fig. 2). We call this the *combined sweep*.

A pseudocode of CbO utilizing the combined sweep is shown in Algorithm 3. We shall only describe the differences from CbO (Algorithm 2). At line 7, we do not recursively call `GenerateFrom` when a set D passes the canonicity test. Instead, we

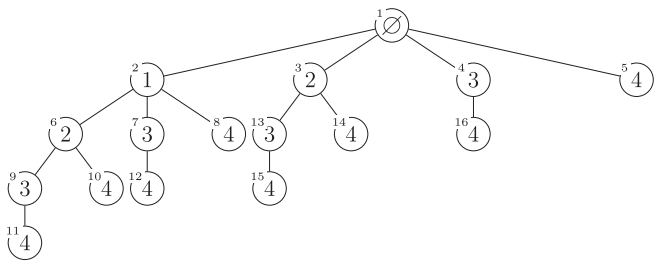


Fig. 2. Combined sweep.

store the arguments for the recursive call in a queue. When the main loop (lines 2–7) terminates, the recursive calls are performed (lines 8–9).

Algorithm 3: Close-by-One with combined sweep

```
def GenerateFrom(A, B, y):
    input : A – extent
            B – intent
            y – last added attribute
1   print(⟨A, B⟩)
2   for i ← y + 1 to n do
3       if i ∉ B then
4           C ← A ∩ i↓
5           D ← C↑
6           if D_i = B_i then
7               PutInQueue(⟨C, D, i⟩)
8   while GetFromQueue(⟨C, D, i⟩) do
9       GenerateFrom(C, D, i)
GenerateFrom(X, X↑, 0)
```

5. Partial closures

The closure in CbO serves two purposes: to check whether the equality $D_i = B_i$ is satisfied and (if it is satisfied) to get the next intent. Andrews [6] observed that one can compute the prefix D_i separately and only then compute the rest of the set D, i.e.

$$_iD = D \cap \{i+1, \ldots, n\},$$

if $D_i = B_i$ holds true.

Splitting the computation of closure into two parts, $_iD$ and D_i, enables us to save computation time in the cases where the second part is not needed, i.e. when $D_i \neq B_i$. This observation is utilized in the algorithm In-Close [6] and its higher versions [7–10].

Computation of D_i We can compute the first part D_i of the intent D as $D_i = C^{\uparrow_i}$, where $\uparrow_i$ is induced by the restriction of the formal context to the first $(i-1)$ attributes.

Computation of $_iD$ In-Close algorithms compute $_iD$ in the same loop in which they visit subtrees of the actual node.

Algorithm 4: In-Close

```
def GenerateFrom(A, B, y):
    input : A – extent
            B – set of attributes
            y – last added attribute
1   for i ← y + 1 to n do
2       C ← A ∩ i↓
3       if A = C then
4           B ← B ∪ {i}
5       else
6           D_i ← C^↑i
7           if B_i = D_i then
8               GenerateFrom(C, B ∪ {i}, i)
9   print(⟨A, B⟩)
GenerateFrom(X, ∅, 0)
```

Partial closures are the primary feature in the In-Close family of algorithms. The pseudocode of In-Close (ver. 1) is shown in Algorithm 4. First, note that the argument B of `GenerateFrom` is not generally an intent. It is a set of attributes which occurred as a result of adding the attribute y to an intent (with exception of initial value $\emptyset$) and which satisfies $B^{\downarrow} = A$. The argument A is an extent, and the algorithm tries to compute new extent C as an intersection of the extent A and the i's column (line 2) as in CbO. If the extent C is equivalent to A (line 3), the attribute i is in $A^{\uparrow}$ and we put it into B (line 4). Otherwise, we compute part D_i of the intent (line 6) and use it to check the canonicity (line 7). If it passes, we recursively call the procedure with the new extent C and with $B \cup \{i\}$ being the base of the corresponding intent (line 8). When the loop is finished, all attributes which are confirmed to be in $A^{\uparrow}$ were added to B. Therefore, B is the intent of A. We can then print it (line 9). The procedure is initially called with full extent X, empty base $\emptyset$ and an invalid last added attribute 0.

In-Close2 differs from the original In-Close only in the sweep direction. It utilizes combined sweep (lines 7 and 9, 10 in Algorithm 5). This allows the algorithm to avoid some unnecessary computations because intent B is finalized before the recursive call. That enables us to skip the main loop for attributes which are already in the finalized intent (line 2).

Algorithm 5: In-Close2

```
    def GenerateFrom(A, B, y):
        input : A – extent
                B – set of attributes
                y – last added attribute
1       for i ← y + 1 to n do
2           if i ∉ B then
3               C ← A ∩ i↓
4               if A = C then
5                   B ← B ∪ {i}
6               else if B_i = C↑_i then
7                   PutInQueue(C, i)
8       print(⟨A, B⟩)
9       while GetFromQueue(C,i) do
10          GenerateFrom(C, B ∪ {i}, i)
    GenerateFrom(X ∅, 0)
```

6. Pruning

While the closures significantly reduce the search space, they are the most expensive operation in CbO. FCbO and In-Close ver. 3, 4, and 5 store information on failed canonicity tests to be used later to avoid some redundant closure computations. We call this technique a *pruning*.[2]

The three pruning techniques exploit the monotony of closure operators:

$$B' \subseteq B \quad \text{implies} \quad B'^{\downarrow\uparrow} \subseteq B^{\downarrow\uparrow} \qquad \text{for all } B, B' \subseteq Y. \tag{7}$$

To properly use the implication (7) for pruning, the algorithms utilize the combined sweep. However, we made an observation that the pruning techniques in the algorithms are not bound to the combined sweep:

Observation 1. To apply pruning in FCbO and In-Close[3] ver. 3, 4, and 5, we can use the right depth-first sweep instead of the combined sweep.

The description of the pruning techniques follows.

6.1. Pruning in FCbO

The following proposition is a consequence of (7).

Proposition 1. Let $B' \subseteq B \subseteq Y$ and $i, j \in Y$. Then

[2] The pruning is also called pre-canonicity test in [31]

[3] However, In-Close algorithms require the combined sweep or left depth-first sweep for the partial closures.

$$j \in (B' \cup \{i\})^{\downarrow\uparrow} \quad \text{implies} \quad j \in (B \cup \{i\})^{\downarrow\uparrow}. \tag{8}$$

More specifically, assume that j in Proposition 1 is an attribute which caused $D' = (B' \cup \{i\})^{\downarrow\uparrow}$ to fail the canonicity test; i.e.

$$j \in B'_i \setminus D'_i.$$

If we take any set $B \supseteq B'$ with $j \notin B$, the attribute j causes $D = (B \cup \{i\})^{\downarrow\uparrow}$ to fail the canonicity test as well.

Specifically, the procedure `GenerateFrom` of FCbO, besides the extent A, intent B, and attribute y, uses an additional argument – a system $\{N^i \mid i \in Y\}$ of sets of attributes. Each N^i is an intent which occurred as $N^i = (B' \cup \{i\})^{\downarrow\uparrow}$ for some intent B' on the path from B to the root, and which failed the canonicity test; or an empty set if such B' does not exists.

As N^i is closed and $i \in N^i$, we have $N^i = (N^i \cup \{i\})^{\downarrow\uparrow}$ and (8) for N^i becomes:

$$j \in N^i \quad \text{implies} \quad j \in (B \cup \{i\})^{\downarrow\uparrow}.$$

Therefore, for $j < i$, the set $D = (B \cup \{i\})^{\downarrow\uparrow}$ fails the canonicity test unless $j \in B$. In short, D fails the canonicity test unless $N^i_i \subseteq B_i$.

Initially, the N^is are empty.

The pseudocode of FCbO is shown in Algorithm 6. We shall describe the differences from CbO with combined sweep (Algorithm 3). The procedure `GenerateFrom` uses the additional argument as described above. It uses it to skip those attributes i which do not satisfy $N^i_i \subseteq B_i$ (line 4). Furthermore, it forms a new system of sets M^i of intents which failed the canonicity test for recursive calls. If D failed the canonicity test, it is stored as M^i (line 10), otherwise M^i is inherited from N^i (line 3).

Algorithm 6: FCbO

```
    def GenerateFrom(⟨A, B⟩, y, {N^j | j ∈ Y}):
        input : A – extent
                B – intent
                y – last added attribute
                N^j s – sets of attributes
1       print(⟨A, B⟩)
2       for i ← y + 1 to n do
3           M^i ← N^i
4           if i ∉ B and N^i_i ⊆ B_i then
5               C ← A ∩ i↓
6               D ← C↑
7               if B_i = D_i then
8                   PutInQueue (⟨C, D, i⟩)
9               else
10                  M^i ← D
11      while GetFromQueue(⟨C, D, i⟩) do
12          GenerateFrom(⟨C, D⟩, i, {M^j | j ∈ Y})
    GenerateFrom(X, X↑, 0, {N^j = ∅ | j ∈ Y})
```

In-Close3 [8] introduces FCbO's pruning into the In-Close family of algorithms. For the sake of completeness, we show its pseudocode in Algorithm 7.

Algorithm 7: In-Close3

```
def GenerateFrom(A, B, y, {N^j | j ∈ Y}):
    input : A – extent
            B – set of attributes
            y – last added attribute
            N^j s – sets of attributes
    for i ← y + 1 to n do
        if i ∉ B and N_i^i ⊆ B_i then
            M^i ← N^i
            C ← A ∩ i↓
            if A = C then
                B ← B ∪ {i}
            else if B_i = C↑_i then
                PutInQueue(C, i)
            else
                M^i ← D
    print(⟨A, B⟩)
    while GetFromQueue(C,i) do
        GenerateFrom(C, B ∪ {i}, i, M)
GenerateFrom(X ∅, 0, {N^j = ∅ | j ∈ Y})
```

6.2. Pruning in In-Close4

The In-Close4 utilizes a weaker but less computationally expensive pruning based on the following proposition.

Proposition 2. Let $B' \subseteq B \subseteq Y$. Then

$$(B' \cup \{i\})^{\downarrow\uparrow} = Y \quad \text{implies} \quad (B \cup \{i\})^{\downarrow\uparrow} = Y. \tag{9}$$

In a formal context which does not contain full rows, we have $Y^{\downarrow} = \varnothing$ and, therefore, (9) becomes

$$(B' \cup \{i\})^{\downarrow} = \varnothing \quad \text{implies} \quad (B \cup \{i\})^{\downarrow} = \varnothing. \tag{10}$$

Specifically, the procedure `GenerateFrom` of In-Close4, uses a single set N of attributes. The elements of N are those $i \in Y$ which caused $(B' \cup \{i\})^{\downarrow}$ to be empty for a previous intent B'. By (10), $(B \cup \{i\})^{\downarrow}$ is empty as well, therefore the algorithm skips it and processes only those which are not in N.

	pruning			cs	pc
	3	4	5		
CbO					
In-Close					×
In-Close2				×	×
In-Close3	×			×	×
In-Close4		×		×	×
In-Close5		×	×	×	×
FCbO	×			×	

Fig. 3. Formal context of CbO-based algorithms, the attributes 3, 4, and 5 represent pruning in In-Close algorithm of corresponding version, 'cs' represents the combined sweep, 'pc' represents the partial closure.

The pseudocode of In-Close4 is shown in Algorithm 8. In contrast to In-Close3 (Algorithm 7), the algorithm uses a single set of attributes N. The set N is used to store the attributes which, when added to the current intent, lead to an empty extent. The attributes in N are skipped in the main loop (line 3). For recursive invocations, N is updated whenever the new extent C is empty (lines 5, 6). Note that M in Algorithm 8 serves as a local copy of the set N.

Algorithm 8: In-Close4

```
def GenerateFrom(A, B, y, N):
    input : A – extent
            B – set of attributes
            y – last added attribute
            N – set of attributes
1   M ← N
2   for i ← y + 1 to n do
3       if i ∉ B and i ∉ N then
4           C ← A ∩ i↓
5           if C = ∅ then
6               M ← M ∪ {i}
7           else if A = C then
8               B ← B ∪ {i}
9           else if B_i = C↑_i then
10              PutInQueue(C, i)
11  print(⟨A, B⟩)
12  while GetFromQueue(C,i) do
13      GenerateFrom(C, B ∪ {i}, i, M)
GenerateFrom(X ∅, 0, ∅)
print(⟨∅, Y⟩)
```

Remark 3. In-Close4 assumes that the input formal context satisfies $Y^{\downarrow} = \emptyset$, or equivalently, that there is no object $x \in X$ satisfying $x^{\uparrow} = Y$. In words, there is no full row in the input formal context. Such objects represent trivial information and they are present in all extents of the formal context. The user can simply remove them. The resulting formal context has the same intents as the original one and extents differ only in the presence of the removed objects.

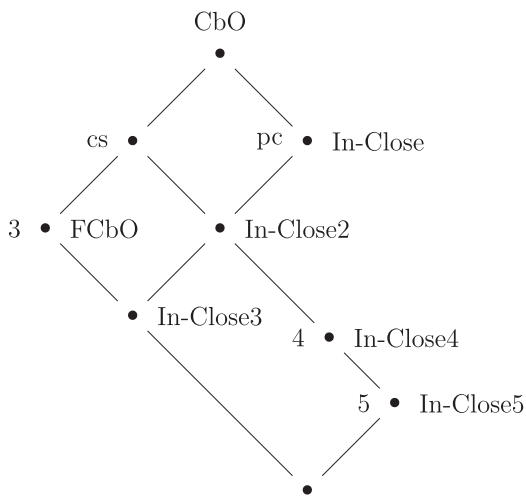


Fig. 4. Concept lattice of the formal context of CbO-based algorithms (Fig. 3). The nodes represent formal concepts, the labels on the right side of nodes are objects, the labels on the left side are attributes. Each formal concept contains those objects which lie on paths going downwards from its node and those attributes which lie on paths going upwards from the node. For example, the node labeled 'In-Close2' corresponds to formal concept ⟨{In–Close2, In–Close3, In–Close4, In–Close5}, {cs, pc}⟩.

The simplified implication (10) allows us to skip computation of $(\cdot)^{\uparrow}$ in (9) for empty extents.

6.3. Pruning in In-Close5

In-Close5 strengthens the pruning of In-Close4 as it additionally stores the attributes i for which $D = (B \cup \{i\})^{\downarrow\uparrow}$ represents a jump outside the subtree of B (unlike FCbO which stores information about attributes for which D represents a jump outside the subtree of $B \cup \{i\}$).

The pseudocode of In-Close5 is shown in Algorithm 9. The only difference from the pseudocode of In-Close4 (Algorithm 8) is a new else-if branch (lines 11,12) which updates N with attributes causing a jump outside the subtree of B.

Algorithm 9: In-Close5

```
def GenerateFrom(A, B, y, N):
    input : A – extent
            B – set of attributes
            y – last added attribute
            N – set of attributes
1   M ← N
2   for i ← y + 1 to n do
3       if i ∉ B and i ∉ N then
4           C ← A ∩ i↓
5           if C = ∅ then
6               M ← M ∪ {i}
7           else if A = C then
8               B ← B ∪ {i}
9           else if B_i = C↑_i then
10              PutInQueue(C, i)
11          else if min(C↑_i − B) < y then
12              M ← M ∪ {i}
13  print(⟨A, B⟩)
14  while GetFromQueue(C,i) do
15      GenerateFrom(C, B ∪ {i}, i)
GenerateFrom(X ∅, 0, ∅)
print(⟨∅, Y⟩)
```

7. Systemization of CbO-based algorithms

In the previous sections, we identified and described five features:

algorithm uses the combined sweep – further denoted by 'cs',
algorithm uses partial closures – further denoted by 'pc',
algorithm uses pruning used in In-Close3 (and FCbO), In-Close4, and In-Close5 – further denoted by 3, 4, and 5, respectively.

Formal context in Fig. 3 shows established algorithms previously described in the literature (and recalled in the previous sections) and the induced concept lattice is depicted in Fig. 4. These seven algorithms were usually compared with others in the papers which introduced them. We compare them also with CbO-based algorithms with other possible sets of features. The formal context in Fig. 5 shows their list and properties and the concept lattice in Fig. 6 shows their hierarchy. We name the algorithms with new sets of features just by their set of features (i.e. intent of the formal context in Fig. 5). For example, the algorithm using combined sweep and pruning 3 and 4 will be named $\{cs, 3, 4\}$-CbO.

The five identified features make 32 possible combinations, however, only 19 of them are considered in this paper (see Fig. 5 for their list and Fig. 6 for their hierarchy):

	pruning 3	pruning 4	pruning 5	cs	pc
CbO					
	×				
	×	×			
		×			
		×	×		
	×	×	×		
				×	
FCbO	×			×	
	×	×		×	
		×		×	
		×	×	×	
	×	×	×	×	
In-Close					×
In-Close2				×	×
In-Close3	×			×	×
	×	×		×	×
In-Close4		×		×	×
In-Close5		×	×	×	×
	×	×	×	×	×

Fig. 5. Context of CbO-based algorithms considered in the present paper.

- First, partial closures require left-first sweep or combined sweep. Pruning requires right-first sweep or combined sweep. That means that an algorithm with partial closures and any pruning must use combined sweep. That makes us dismiss 7 combinations.
- Second, pruning of In-Close5 already contains the pruning of In-Close4, and therefore, we do not consider feature 5 without feature 4. That allows us to dismiss another 6 combinations.

8. Experimental evaluation

We have implemented all algorithms described in Fig. 5 to evaluate their performance.

8.1. Remarks on test and algorithm implementations

All algorithms have been implemented in C++ from scratch using the same programming style, data structures, and code base to make the comparison as fair and relevant as possible. We have directly followed the pseudo-codes as discussed in the previous sections, with the exception of {cs}-CbO, {cs, 4}-CbO, {cs, 5}-CbO. For these algorithms, it turns out to be efficient to compute all closures first, i.e. in a single pass through the context. Interestingly, our experiments show that this is not an efficient approach for {cs, 3}-CbO, {cs, 3, 4}-CbO. In the interest of fairness, we have avoided low-level optimizations that could favor any particular algorithm. Thus, our implementation may not be as efficient as well-tuned implementations of algorithms published by their authors.

To represent formal contexts and set of attributes (e.g. intents), bit vectors have been used since it allows efficient implementation of operations used in the CbO algorithms. This means, heavily used operations like intersections of two sets or canonicity tests can be easily compiled to CPU instructions. An array of integers was used to represent extents.

All experiments have been performed on a computer with 64 GiB RAM, two Intel Xeon CPU E5-2680 v2 (at 2.80 GHz), Debian Linux 10, and GNU GCC 8.3.0. Measurements involving real datasets have been taken 10× and the mean value is pre-

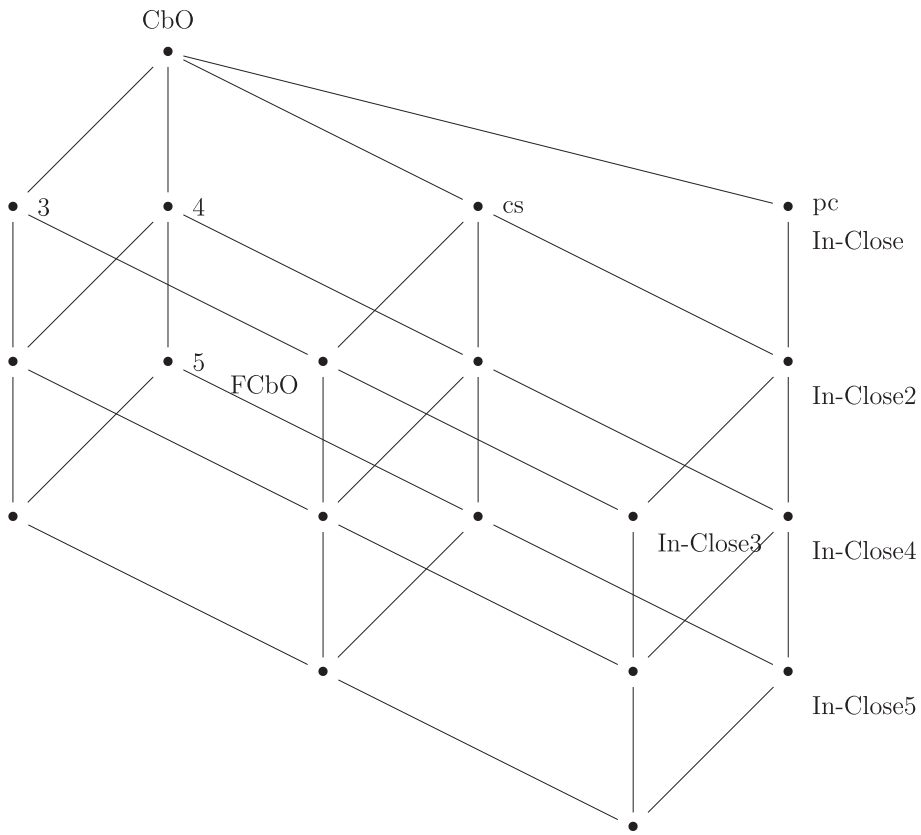


Fig. 6. Concept lattice of CbO-based algorithms considered in the present paper.

sented. In case of artificial datasets, for each combination of properties, 10 unique datasets have been created and the average computation time is used.

As [26] points out, the order of attributes of a formal context matters. Namely, reordering of attributes w.r.t. their support (i.e. $|j^{\downarrow}|$ where $j \in Y$) can reduce the number of computed closures. Therefore, we consider only contexts with ordered attributes. As a consequence, the presented times do not include the time of the preprocessing step that reorders attributes.

8.2. Real-world datasets

The first set of experiments uses datasets that are commonly used to evaluate FCA algorithms. The datasets are from the UCI Machine Learning Repository [19] (bcw, mushrooms, anon. web., tic-tac-toe, votes), FIMI Repository[4] (T10I4D100K), from [20] (americas large), and our own dataset (debian tags)[5]. Properties of these datasets are shown in Table 1.

Time spent to enumerate all formal concepts is presented in Table 2 (values in milliseconds). To provide better insight into the results, Table 3 presents the same values but uses a time to minimal time ratio. This means, the time of each algorithm to the time of the fastest algorithm (lower values are better).

8.2.1. Observations

- These results indicate that {3, 4}-CbO seems to be the fastest algorithm, however the differences are quite small.
- Interestingly, the algorithms {3, 4, cs}-CbO and {3, 4, cs, pc}-CbO contain more speed-up features that are expected to improve the runtimes. However, at the end, their overhead slows the algorithms down. Apparently, the maintenance of a queue and the extra loop in the algorithms has noticeable overhead.
- Further, it shows that it is reasonable to combine pruning 3 and 4 but not pruning 3, 4, and 5. The pruning from In-Close4 uses very simple conditions to determine which attributes should be skipped; hence it can save some CPU clock-cycles. This does not hold for the In-Close5 pruning, which uses a more complex condition determining attributes to skip.

[4] http://fimi.uantwerpen.be/data/
[5] http://phoenix.inf.upol.cz/ krajcap/data/datasets.tgz

Table 1
Properties of used datasets

dataset	rows	attrs	crosses	density (%)	concepts
americas large	10,127	3484	185,294	0.52	36,991
bcw	683	90	6,830	11.11	9,569
debian tags	14,315	475	67,672	0.99	38,977
anon. web.	32,710	284	98,653	1.06	129,009
mushrooms	8,124	118	186,852	19.49	238,710
T10I4D100K	100,000	869	1,010,228	1.16	2,347,376
tic-tac-toe	958	28	9,580	35.71	59,505
votes	435	49	7,395	34.69	264,035

- Interestingly, the weakest pruning technique used in {4}-CbO and In-Close4 provides quite competitive results for all used datasets but *americas large*.

8.3. Synthetic datasets

Experiments with real datasets indicate that there are often only subtle differences among the best CbO algorithms. These differences may stem from the different properties of datasets. Note that selected datasets may be inappropriate for the given algorithm, for example, the dataset maybe too small or too dense to be processed efficiently with a particular algorithm.

We have generated artificial datasets having varying properties like number of objects or attributes to investigate the behavior of the discussed algorithms more thoroughly. We considered two types of data one may encounter in FCA:

1. transactional data, i.e., data used in market-basket analysis or recommender systems; for this purpose IBM Quest Synthetic Data Generator has been used.
2. questionnaire-like data, i.e. context consists of attribute groups where each object has exactly one attribute from each group. Note that similar properties have datasets created with nominal scaling. Therefore, we call these datasets *nominal*. We proposed our own tool to generate this type of data.

Table 2
Runtimes (in milliseconds) of all considered CbO-based algorithms for real-word datasets.

dataset	{} (CbO)	{3}	{3, 4}	{4}	{4, 5}	{3, 4, 5}
americas large	5,548	833	844	4,959	1,673	994
bcw	29.5	12.9	**9.7**	12.9	11.8	12.4
debian tags	499.5	96.8	**79.6**	80.9	92.8	90.8
anon. web.	1,575.8	147.5	**137.2**	143.4	168.5	162.0
mushrooms	3,300.8	321.5	**307.8**	412.5	340.4	**308.6**
T10I4D100K	275,292	15,724	11,250	10,917	13,067	13,301
tic-tac-toe	93.0	29.6	**28.4**	30.9	31.0	29.5
votes	420.7	141.2	**137.5**	170.2	155.9	143.4
	{cs}	{3,cs} (FCbO)	{3, 4,cs}	{4,cs}	{4,5,cs}	{3,4,5,cs}
americas large	6,093	772	**752**	5,806	1,934	999
bcw	16.0	11.8	12.1	15.9	15.1	12.7
debian tags	169.4	97.6	81.6	118.0	131.8	93.2
anon. web.	195.2	147.5	140.5	181.5	206.5	168.1
mushrooms	632.6	319.1	**308.9**	523.5	440.8	314.5
T10I4D100K	25,330	13,473	10,295	18,349	19,474	13,151
tic-tac-toe	41.4	30.0	29.6	37.1	36.6	29.7
votes	220.0	144.8	144.6	200.0	190.9	147.0
{pc} (In-Close)	{cs,pc} (In-Close2)	{3,cs,pc} (In-Close3)	{3, 4,cs,pc}	{4,cs,pc} (In-Close4)	{4,5,cs,pc} (In-Close5)	{3,4,5,cs,pc}
americas large	5,712	5,762	926	908	5,655	1,951	1,060
bcw	12.9	12.9	13.5	12.0	11.5	14.5	12.8
debian tags	92.8	101.2	105.2	86.0	82.6	103.2	97.7
anon. web.	139.7	149.6	161.6	151.1	143.1	187.5	177.0
mushrooms	578.5	469.8	328.2	328.1	398.0	371.0	325.7
T10I4D100K	12,370	13,143	15,077	11,018	**9,864**	13,383	13,464
tic-tac-toe	30.3	31.9	32.9	32.4	29.0	34.5	32.8
votes	161.0	172.4	156.2	155.1	157.4	169.4	156.8

Table 3
Time to minimal time ratios of all considered CbO-based algorithms for real-word datasets.

dataset	{} (CbO)	{3}	{3, 4}	{4}	{4, 5}	{3, 4, 5}
americas large	7.38	1.11	1.12	6.60	2.23	1.32
bcw	3.04	1.33	1	1.33	1.21	1.28
debian tags	6.27	1.22	1	1.02	1.17	1.14
anon. web.	11.48	1.08	1	1.04	1.23	1.18
mushrooms	10.72	1.04	1	1.34	1.11	1.00
T10I4D100K	27.91	1.59	1.14	1.11	1.32	1.35
tic-tac-toe	3.28	1.05	1	1.09	1.09	1.04
votes	3.06	1.03	1	1.24	1.13	1.04
average	9.14	1.18	1.03	1.85	1.31	1.17
	{cs}	{3,cs} (FCbO)	{3, 4,cs}	{4,cs}	{4,5,cs}	{3,4,5,cs}
americas large	8.11	1.03	1	7.72	2.57	1.33
bcw	1.65	1.22	1.25	1.64	1.56	1.31
debian tags	2.13	1.23	1.03	1.48	1.66	1.17
anon. web.	1.42	1.07	1.02	1.32	1.50	1.23
mushrooms	2.06	1.04	1.00	1.70	1.43	1.02
T10I4D100K	2.57	1.37	1.04	1.86	1.97	1.33
tic-tac-toe	1.46	1.06	1.04	1.31	1.29	1.05
votes	1.60	1.05	1.04	1.45	1.39	1.07
average	2.62	1.13	1.05	2.31	1.67	1.19
{pc} (In-Close)	{cs,pc} (In-Close2)	{3,cs,pc} (In-Close3)	{3, 4,cs,pc}	{4,cs,pc} (In-Close4)	{4,5,cs,pc} (In-Close5)	{3,4,5,cs,pc}
americas large	7.60	7.67	1.23	1.21	7.52	2.60	1.41
bcw	1.33	1.33	1.39	1.24	1.18	1.49	1.33
debian tags	1.17	1.27	1.32	1.08	1.04	1.30	1.23
anon. web.	1.02	1.09	1.18	1.10	1.04	1.37	1.29
mushrooms	1.88	1.53	1.07	1.07	1.29	1.21	1.06
T10I4D100K	1.25	1.33	1.53	1.12	1	1.36	1.37
tic-tac-toe	1.07	1.13	1.16	1.14	1.02	1.22	1.16
votes	1.17	1.25	1.14	1.13	1.14	1.23	1.14
average	2.06	2.08	1.25	1.13	1.91	1.47	1.25

To explore how features of algorithms affect performance, we have created a set of charts depicting runtime improvement w.r.t. a selected baseline algorithm. The complete set of charts is available as a supplementary material in the form of a single standalone HTML page. A description of this page is provided in B.

We show effect of isolated features first. Then, since the initial observations presented in Table 2 indicate that algorithms with the pruning 3, pruning 4, and In-Close2 family provide good results, we shall discuss their behavior more thoroughly.

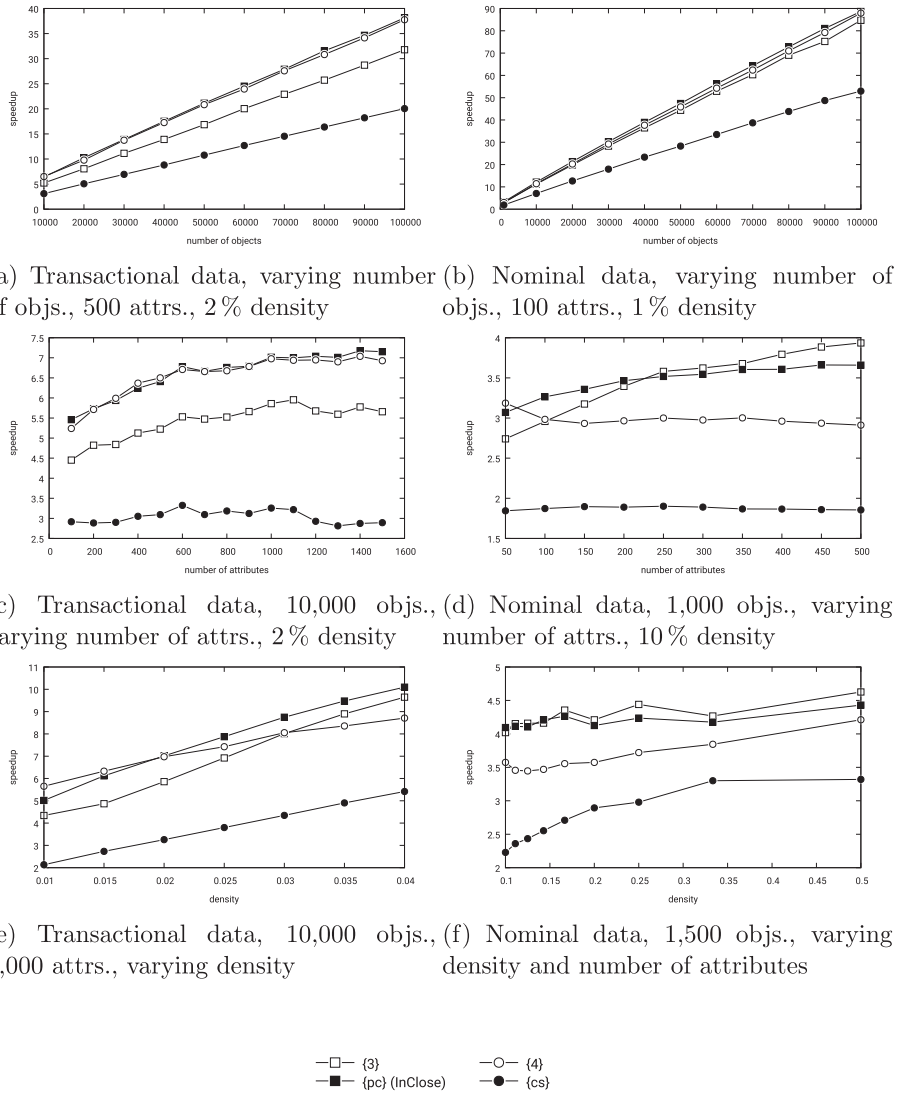
8.4. Observations on algorithms with a single feature

Each feature (combined sweep, partial closures, pruning 3 and 4) improves performance, as shows Fig. 7. Their effect tends to grow or maintain performance with the increasing size of data. Note that the combined sweep is not generally considered as a performance enhancing feature. However, our experiments indicate that it brings a considerable performance improvement. As Fig. 7 shows, its effect is non-negligible, even though, not as significant as that of the partial closure or pruning techniques. In essence, the combined sweep order improves data locality, this means, actively used data (e.g. specific portions of the context) tend to reside in the CPU cache and can be accessed faster.

8.5. Observations on algorithms with pruning 3

For transactional data, three algorithms with pruning 3 seem to provide reasonable improvements, namely, {3,4}-CbO, {3,4,cs}-CbO, and {3,4,cs,pc}-CbO. Their performance seems to be higher and does not change significantly w.r.t. to increasing number of object or attributes, as shown in Fig. 8 (a) and (c). However, the effect of enhancements declines with the increasing density of data. From this point of view, FCbO and In-Close3 provide slower but steady results, see Fig. 8 (e).

For nominal data, the mentioned algorithms are also amongst the best algorithms, but the differences are not significant, see Fig. 8 (b), (d), and (f). Additionally, FCbO seems to be a good algorithm for this type of data.



(a) Transactional data, varying number of objs., 500 attrs., 2 % density

(b) Nominal data, varying number of objs., 100 attrs., 1 % density

(c) Transactional data, 10,000 objs., varying number of attrs., 2 % density

(d) Nominal data, 1,000 objs., varying number of attrs., 10 % density

(e) Transactional data, 10,000 objs., 1,000 attrs., varying density

(f) Nominal data, 1,500 objs., varying density and number of attributes

Fig. 7. Performance improvements w.r.t. to the CbO algorithm.

8.5.1. Observations on algorithms with pruning 4

In the family of algorithms with pruning 4, four algorithms, {3, 4, cs, pc}-CbO, {3, 4}-CbO, {3, 4, cs}-CbO, and {4, cs, pc}-CbO, dominate. All these algorithms provide better results than the basic pruning 4 and maintain performance w.r.t. increasing number of objects and density. Furthermore, they improve or maintain performance with an increasing number of attributes. This seems to hold for both nominal and transactional data, see Fig. 9 for details.

8.6. Observations on algorithms which extend In-Close2

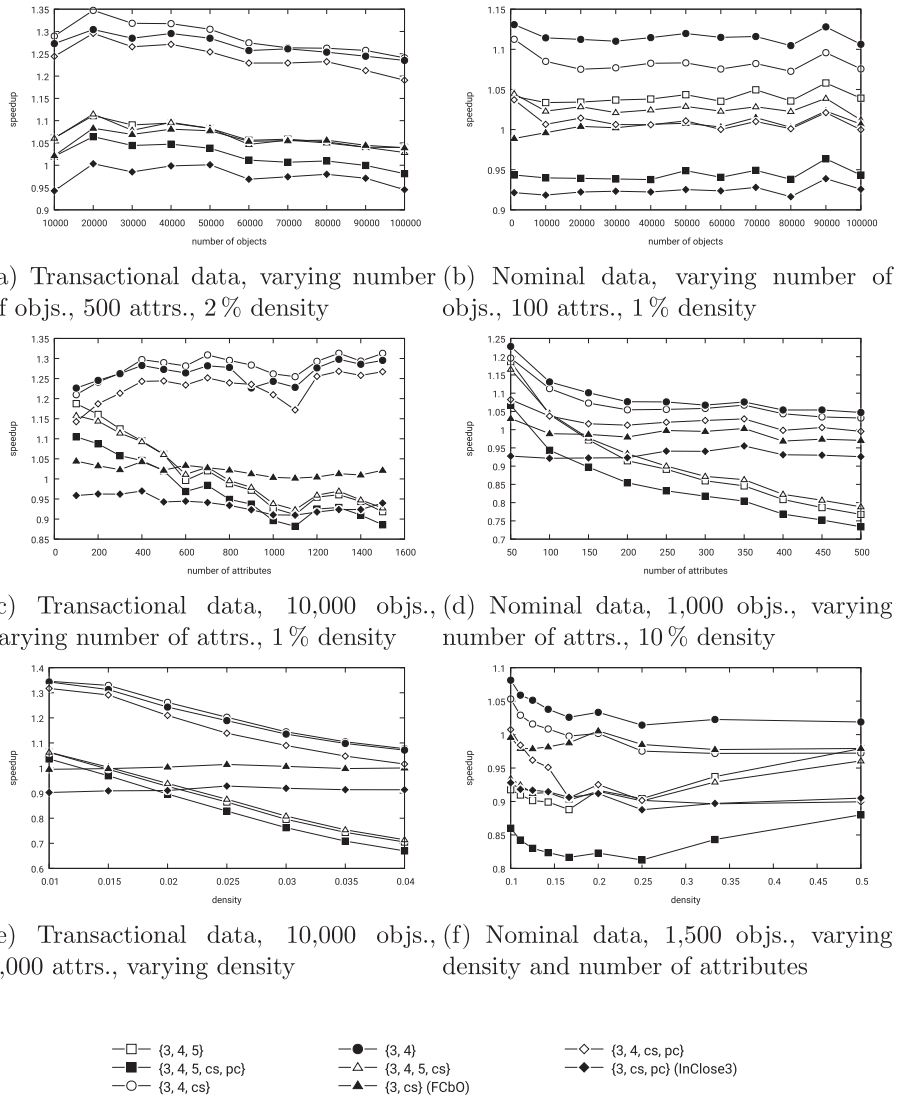
In the In-Close2 family, for both types of datasets, we observe that In-Close4 and {3, 4, cs, pc}-CbO provide the best performance. While {3, 4, cs, pc}-CbO performs better for higher numbers of attributes, see Fig. 10.

9. Conclusions

9.1. Summary

We extracted speed-up features from the CbO-based algorithms, performed a thorough experimental comparison of almost all their combinations, and made numerous observations.

The results which we consider to be most significant are:



(a) Transactional data, varying number of objs., 500 attrs., 2 % density

(b) Nominal data, varying number of objs., 100 attrs., 1 % density

(c) Transactional data, 10,000 objs., varying number of attrs., 1 % density

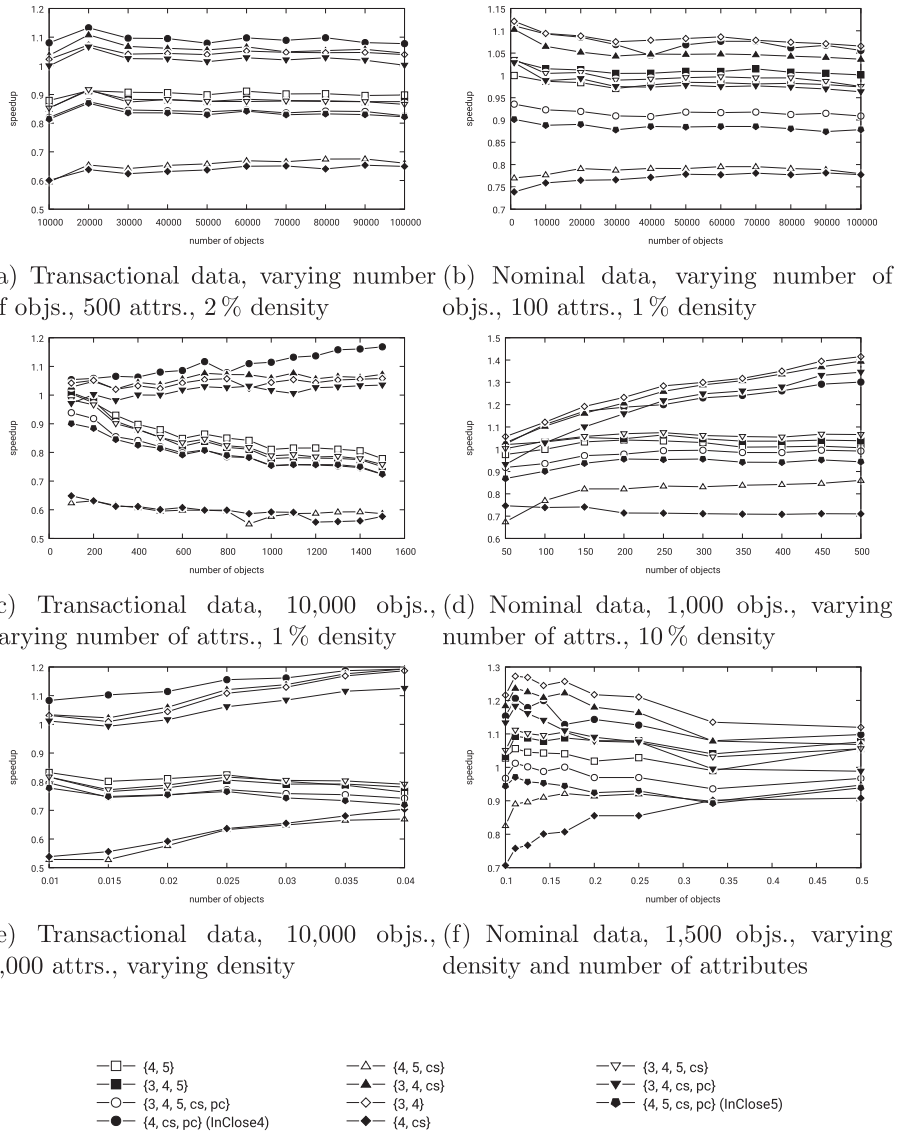
(d) Nominal data, 1,000 objs., varying number of attrs., 10 % density

(e) Transactional data, 10,000 objs., 1,000 attrs., varying density

(f) Nominal data, 1,500 objs., varying density and number of attributes

Fig. 8. Performance improvements w.r.t. basic algorithm with pruning 3.

- **Compatibility of left depth-first sweep and pruning techniques** (Observation 1): this allows the application of pruning in Map-Reduce or similar framework (as already mentioned in Section 1). However, as another crucial consequence, we can use the pruning techniques while computing closed sets in lectic order [21]. Therefore, we can use CbO-based algorithms with pruning to compute pseudo-intents. This means, that one can design faster algorithms for computing attribute implications — another important outcome of FCA. Our preliminary tests indicate promising results on the increase of efficiency.
- **Fastest algorithms for enumeration of formal concepts/closed sets**: existing algorithms were designed mostly in an ad hoc manner by combining promising features. By extracting independent features and combining them back together, we were able to study the contribution of each feature to the algorithm performance. This has two important outcomes. Firstly, we identified new and faster CbO-algorithms than the existing ones. The combination of pruning 3 and 4 (possibly with the combined sweep and partial closures) seems to be very fruitful. All the algorithms that we evaluated as fastest exploit this combination (see Appendix A). At present, these algorithms are arguably the fastest algorithms for the computation of formal concepts in general – i.e. not limited to the class of CbO-based algorithms. Secondly, we observed that a cumulation of speed-up features can lead to a slowdown as their total overhead cost often exceeds total benefits.



(a) Transactional data, varying number of objs., 500 attrs., 2 % density

(b) Nominal data, varying number of objs., 100 attrs., 1 % density

(c) Transactional data, 10,000 objs., varying number of attrs., 1 % density

(d) Nominal data, 1,000 objs., varying number of attrs., 10 % density

(e) Transactional data, 10,000 objs., 1,000 attrs., varying density

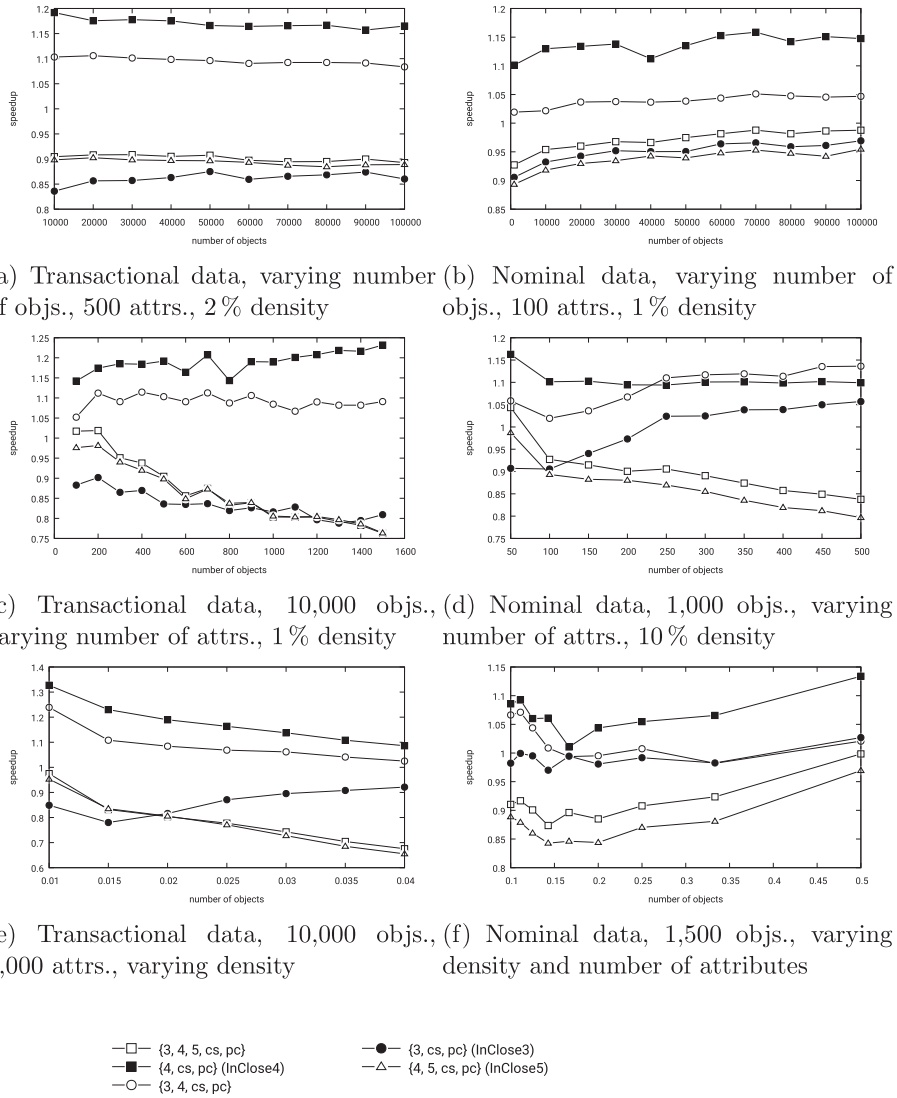
(f) Nominal data, 1,500 objs., varying density and number of attributes

Fig. 9. Performance improvements w.r.t. basic algorithm with pruning 4.

9.2. Future research

We see the following points as our future research areas:

- We will perform an analytical comparison of CbO-based algorithm with another significant group of algorithms for enumerating closed sets – consensus-based algorithms like SPIC and SPAN used in logical analysis of data [5,3]. In [22], we claimed that CbO-based algorithms are an interesting alternative to the consensus-based algorithms. Our preliminary performance tests presented therein show superiority of the CbO-based algorithms, however a thorough analysis is still to be done. The pruning techniques (which are not applicable in consensus-based algorithms) could represent a deciding argument in this analysis.
- We will study possible uses of the pruning techniques in non-CbO-based algorithms. Our preliminary results indicate that pruning 4 can be used to improve performance of algorithms for concept lattice construction.



(a) Transactional data, varying number of objs., 500 attrs., 2 % density

(b) Nominal data, varying number of objs., 100 attrs., 1 % density

(c) Transactional data, 10,000 objs., varying number of attrs., 1 % density

(d) Nominal data, 1,000 objs., varying number of attrs., 10 % density

(e) Transactional data, 10,000 objs., 1,000 attrs., varying density

(f) Nominal data, 1,500 objs., varying density and number of attributes

Fig. 10. Performance improvements w.r.t. In-Close2.

CRediT authorship contribution statement

Jan Konecny: Writing - original draft, Writing - review & editing, Funding acquisition. **Petr Krajča:** Conceptualization, Software, Visualization.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors acknowledge support by the grant JG 2019 of Palacký University Olomouc, No.JG_2019_008.

Appendix A. Pseudocodes of the fastest algorithms

Here we provide standalone descriptions of the three algorithms which performed best in our experimental evaluation, namely $\{3, 4\}$-CbO, $\{3, 4$,cs$\}$-CbO, and $\{3, 4$,cs,pc$\}$-CbO. We put this to an appendix as their pseudocodes can be quite easily derived from those in Sections 3,4,5, and 6.

A.1. Algorithm $\{3,4\}$-CbO

The algorithm $\{3,4\}$-CbO (see Algorithm 10) is given by the recursive procedure `GenerateFrom`, which accepts five arguments:

- A, B – respectively the extent and intent of the present formal concept $\langle A, B\rangle$;
- y – the auxiliary argument to store the last attribute added to B;
- $\{M^j \mid j \in Y\}$ and N – information for pruning 3 and pruning 4, respectively.

It performs the following:

(line 1) prints the present formal concept $\langle A, B\rangle$.
(line 2) makes a copy of information on empty extents for pruning 4.
(lines 3–14) are the main loop going through attributes i, higher than the last added attribute, in descending order (right-first sweep through the search tree is necessary for both used pruning techniques, unless combined sweep is used).
(line 4) makes a copy of information on failed canonicity tests for attribute i for pruning 3.
(line 5) skips attributes i which are already in the present intent B or are pruned by pruning 3 or pruning 4.
(line 6) computes new extent C as an intersection of the present extent A and i's column.
(line 7–8) update the information on empty extents for pruning 4 if C is empty.
(line 10) computes the new intent D as $C^{\uparrow}$.
(line 11) performs the canonicity test.
(line 12) recursively calls `GenerateFrom` for the new extent C and intent D, last added attribute i and information for pruning 3 and 4, if D passes the canonicity test.
(lines 13–14) update the information on failed canonicity test for pruning 3 if D does not pass the canonicity test.

The procedure is initially called with formal concept $\langle X, X^{\uparrow}\rangle$, induced by an empty set of attributes, with 0 indicating that no attribute was added, and with empty information for both pruning techniques.

Algorithm 10: $\{3,4\}$-CbO

```
def GenerateFrom(⟨A, B⟩, y, {N^j | j ∈ Y}, N):
    input : ⟨A, B⟩ – formal concept
            y – last added attribute
            N^j s – sets of attributes
            N – set of attributes
1   print(⟨A, B⟩)
2   M ← N
3   for i ← n down to y + 1 do
4       M^i ← N^i
5       if i ∉ B and i ∉ N and N_i^i ⊆ B_i then
6           C ← A ∩ i^↓
7           if C = ∅ then
8               M ← M ∪ {i}
9           else
10              D ← C^↑
11              if B_i = D_i then
12                  GenerateFrom(⟨C, D⟩, i, {M^j | j ∈ Y}, M)
13              else
14                  M^i ← D
GenerateFrom(⟨X, X^↑⟩, 0, {N^j = ∅ | j ∈ Y}, ∅)
```

A.2. Algorithm $\{3,4,cs\}$-CbO

The algorithm $\{3,4,cs\}$-CbO (see Algorithm 11) is given by the recursive procedure `GenerateFrom`, which accepts five arguments:

- A, B – respectively the extent and intent of the present formal concept $\langle A, B\rangle$;
- y – the auxiliary argument to store the last attribute added to B;
- $\{M^j \mid j \in Y\}$ and N – information for pruning 3 and 4, respectively.

It performs the following:

(line 1) prints the present formal concept $\langle A, B\rangle$.
(line 2) makes a copy of information on empty extents for pruning 4.
(lines 3–14) are the main loop going through attributes i, higher than the last added attribute, in ascending order (presented as left-first sweep, however it could be right-first sweep as well).
(line 4) makes a copy of information on failed canonicity tests for attribute i for 3 pruning.
(line 5) skips attributes i which are already in the present intent B or are pruned by pruning 3 or 4.
(line 6) computes new extent C as an intersection of the present extent A and i's column.
(line 7–8) update the information on empty extents for pruning 4 if C is empty.
(line 10) computes the new intent D as $C^{\uparrow}$.
(line 11) performs the canonicity test.
(line 12) stores arguments for later recursive calls in a queue.
(lines 13–14) update the information on failed canonicity test for pruning 3 if D does not pass the canonicity test.
(lines 15–16) recursively call the procedure for all sets of arguments stored in the queue; emptying the queue.

The procedure is initially called with formal concept $\langle X, X^{\uparrow}\rangle$, induced by an empty set of attributes, with 0 indicating that no attribute was added, and with empty information for both pruning techniques.

Algorithm 11: $\{3,4,cs\}$-CbO

```
def GenerateFrom(A, B, y, {N^j | j ∈ Y}, N):
    input : A – extent
            B – intent
            y – last added attribute
            N^j s – sets of attributes
            N – set of attributes
1   print(⟨A, B⟩)
2   M ← N
3   for i ← y + 1 to n do
4       M^i ← N^i
5       if i ∉ B and i ∉ N and N_i^j ⊆ B_i then
6           C ← A ∩ i^↓
7           if C = ∅ then
8               M ← M ∪ {i}
9           else
10              D ← C^↑
11              if B_i = D_i then
12                  PutInQueue (⟨C, D, i⟩)
13              else
14                  M^i ← D
15      while GetFromQueue(⟨C, D, i⟩) do
16          GenerateFrom(⟨C, D⟩, i, {M^j | j ∈ Y}, M)
GenerateFrom(X, X^↑, 0, {N^j = ∅ | j ∈ Y}, ∅)
```

A.3. Algorithm $\{3,4,cs,pc\}$-CbO

The algorithm $\{3,4,cs,pc\}$-CbO (see Algorithm 12) is given by the recursive procedure `GenerateFrom`, which accepts five arguments:

- A, B – respectively the extent and intent of the present formal concept $\langle A, B\rangle$;
- y – the auxiliary argument to store the last attribute added to B;
- $\{M^j \mid j \in Y\}$ and N – information for pruning 3 and 4, respectively.

(line 1) makes a copy of information on empty extents for pruning 4.
(lines 2–11) are the main loop going through attributes i, higher than the last added attribute, in ascending order (presented as left-first sweep, however it could be right-first sweep as well).
(line 3) makes a copy of information on failed canonicity tests for attribute i for 3 pruning.
(line 4) skips attributes i which are already in the present intent B or are pruned by pruning 3 or 4.
(line 5) computes new extent C as an intersection of the present extent A and i's column.
(line 6–7) update the information on empty extents for pruning 4 if C is an empty set.
(line 8–9) are the partial closing: if the new extent C is equal to the previous extent A, the attribute i is added to A's intent B.
(line 10) performs the canonicity test.
(line 11) stores arguments for later recursive calls in a queue.
(line 12) prints the formal concept $\langle A, B\rangle$ as computation of B is now finished.
(lines 13–14) recursively call the procedure for all sets of arguments stored in the queue; emptying the queue.

Algorithm 12: $\{3,4, cs, pc\}$-CbO

```
def GenerateFrom(A, B, y, {N^j | j ∈ Y}, N):
    input : A – extent
            B – set of attributes
            y – last added attribute
            N^j s – sets of attributes
            N – set of attributes
1   M ← N
2   for i ← y + 1 to n do
3       M^i ← N^i
4       if i ∉ B and i ∉ N and N_i^i ⊆ B_i then
5           C ← A ∩ i↓
6           if C = ∅ then
7               M ← M ∪ {i}
8           else if A = C then
9               B ← B ∪ {i}
10          else if B_i = C↑_i then
11              PutInQueue(C, i)
12  print(⟨A, B⟩)
13  while GetFromQueue(C,i) do
14      GenerateFrom(C, B ∪ {i}, i, {M^j | j ∈ Y}, M)
GenerateFrom(X ∅, 0, {N^j = ∅ | j ∈ Y}, ∅)
print(⟨∅, Y⟩)
```

Appendix B. Webpage with performance comparison of CbO-based algorithms

We provide a complete set of data we collected. This data describe performance changes when selected features of CbO algorithms are applied. This serves as a supplementary material for Section 8. For convenience, the data are presented as a single interactive HTML page (see Fig. B11).

The HTML page contains five inputs controlling the output:

- *baseline algorithm* – an algorithm serving as a basis for performance comparison,
- *compared algorithms* – a collection of algorithms that are compared to the baseline algorithm. Two options are available:

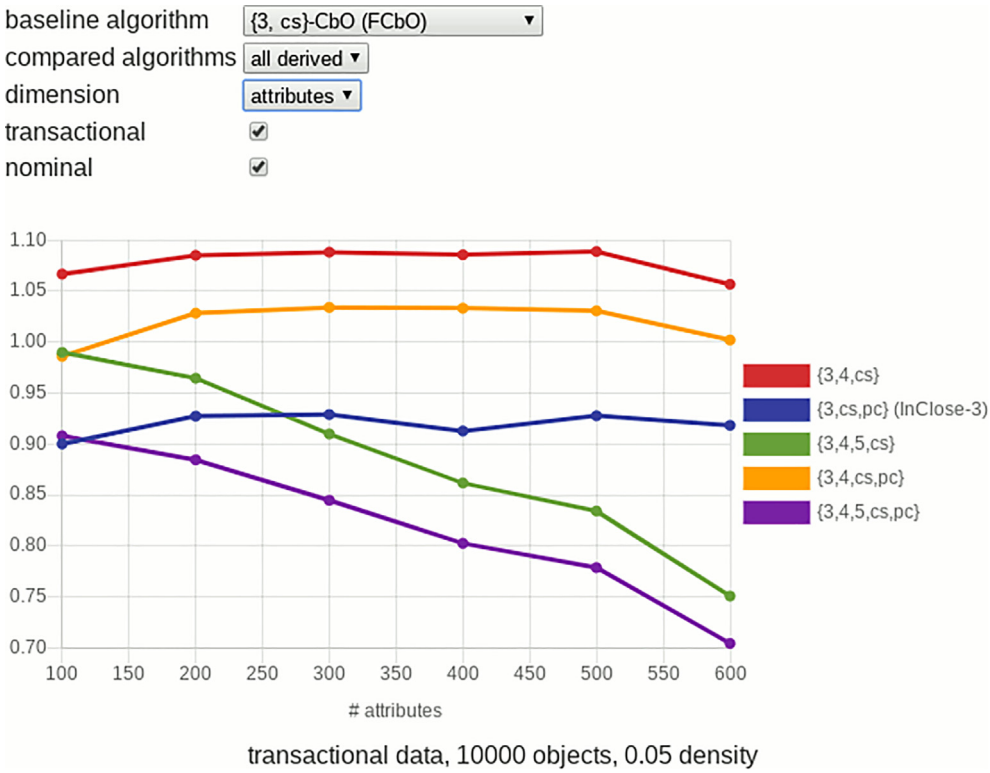


Fig. B11. Part of the interactive HTML page with supplementary charts.

1. all derived – all algorithms sharing the same features as the baseline algorithm (i.e. are derived from the baseline algorithm),
2. neighbors – algorithms that are derived from the baseline algorithm and have exactly one extra feature (i.e. are lower neighbors in the lattice of CbO-algorithms, Fig. 6).

- *dimension* – varying dimension of used datasets (number of objects, number of attributes, density),
- *nominal, transactional* – type of used datasets (see Section 8 for the description).

Based on the input, the HTML page shows charts depicting performance changes w.r.t. baseline algorithm, in other words, the ratio of the running time of the baseline algorithm to other algorithms.

The output charts are interactive and one can hide algorithms or use a pointing device to see detailed values.

References

[1] Cristina Alcalde, Ana Burusco, Multivalued contexts associated with criteria, Int. J. Gen. Syst. 47 (2) (2018) 118–136.
[2] Cristina Alcalde, Ana Burusco, Reduction of the size of L-fuzzy contexts. A tool for differential diagnoses of diseases, Int. J. Gen. Syst. 48 (7) (2019) 692–712.
[3] Gabriela Alexe, Sorin Alexe, Tibérius O Bonates, Alexander Kogan, Logical analysis of data–the vision of Peter L. Hammer, Ann. Math. Artif. Intell. 49 (1–4) (2007) 265–312.
[4] Gabriela Alexe, Sorin Alexe, Yves Crama, Stephan Foldes, Peter L Hammer, and Bruno Simeone. Consensus algorithms for the generation of all maximal bicliques. Discrete Applied Mathematics, 145(1), 11–21, 2004..
[5] Gabriela Alexe, Peter L Hammer, Spanned patterns for the logical analysis of data, Discrete Appl. Math. 154 (7) (2006) 1039–1049.
[6] Simon Andrews, In-close, a fast algorithm for computing formal concepts, in: International Conference on Conceptual Structures, Springer, 2009.
[7] Simon Andrews, In-close2, a high performance formal concept miner, in: International Conference on Conceptual Structures, Springer, 2011, pp. 50–62.
[8] Simon Andrews, A 'best-of-breed' approach for designing a fast algorithm for computing fixpoints of Galois connections, Inf. Sci. 295 (2015) 633–649.
[9] Simon Andrews, in: Making use of empty intersections to improve the performance of CbO-type algorithms In International Conference on Formal Concept Analysis, Springer, 2017, pp. 56–71.
[10] Simon Andrews. A new method for inheriting canonicity test failures in close-by-one type algorithms, 2018..
[11] Eduard Bartl, Jan Konecny, L-concept analysis with positive and negative attributes, Inf. Sci. 360 (2016) 96–111.
[12] Eduard Bartl, Jan Konecny, L-concept lattices with positive and negative attributes: Modeling uncertainty and reduction of size, Inf. Sci. 472 (2019) 163–179.
[13] Radim Belohlavek, Fuzzy Relational Systems: Foundations and Principles, Kluwer Academic Publishers, Norwell, USA, 2002.
[14] Radim Belohlavek, Vilem Vychodil, Discovery of optimal factors in binary data via a novel method of matrix decomposition, J. Comput. Syst. Sci. 76 (1) (2010) 3–20.
[15] Ana Burusco, Ramón Fuentes-González, Construction of the L-fuzzy concept lattice, Fuzzy Sets Syst. 97 (1) (1998) 109–114.
[16] Claudio Carpineto, Giovanni Romano, Concept Data Analysis: Theory and Applications, John Wiley & Sons, 2004.

[17] Maria Eugenia Cornejo, Jesús Medina, Eloísa Ramírez-Poussa, Characterizing reducts in multi-adjoint concept lattices, Inf. Sci. 422 (2018) 364–376.
[18] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In 6th Symposium on Operating System Design and Implementation (OSDI 2004), San Francisco, California, USA, December 6–8, 2004, pages 137–150, 2004..
[19] Dheeru Dua and Casey Graff. UCI machine learning repository, 2017..
[20] Alina Ene, William G. Horne, Nikola Milosavljevic, Prasad Rao, Robert Schreiber, and Robert Endre Tarjan. Fast exact and heuristic methods for role minimization problems. In Indrakshi Ray and Ninghui Li, editors, 13th ACM Symposium on Access Control Models and Technologies, SACMAT 2008, Estes Park, CO, USA, June 11–13, 2008, Proceedings, pages 1–10. ACM, 2008..
[21] Bernard Ganter, Rudolf Wille, Formal Concept Analysis – Mathematical Foundations, Springer, 1999.
[22] Radek Janostik, Jan Konecny, Petr Krajča, Interface between logical analysis of data and formal concept analysis, Eur. J. Oper. Res. 284 (2) (2020) 792–800.
[23] David S Johnson, Mihalis Yannakakis, Christos H Papadimitriou, On generating all maximal independent sets, Information Processing Letters 27 (3) (1988) 119–123.
[24] Jan Konecny, Attribute implications in L-concept analysis with positive and negative attributes: Validity and properties of models, Int. J. Approx. Reason. 120 (2020) 203–215.
[25] Jan Konecny and Petr Krajca. Pruning in map-reduce style cbo algorithms. In Mehwish Alam, Tanya Braun, and Bruno Yun, editors, Ontologies and Concepts in Mind and Machine – 25th International Conference on Conceptual Structures, ICCS 2020, Bolzano, Italy, September 18–20, 2020, Proceedings, volume 12277 of Lecture Notes in Computer Science, pages 103–116. Springer, 2020..
[26] Petr Krajca, Jan Outrata, Vilem Vychodil, Parallel algorithm for computing fixpoints of Galois connections, Annals of Mathematics and Artificial Intelligence 59 (2) (2010) 257–272.
[27] Sergei O Kuznetsov. A fast algorithm for computing all intersections of objects from an arbitrary semilattice. Nauchno-Tekhnicheskaya Informatsiya Seriya 2-Informatsionnye Protsessy i Sistemy, (1):17–20, 1993..
[28] Sergei O. Kuznetsov, Sergei Obiedkov, Comparing performance of algorithms for generating concept lattices, J. Experimental Theor. Artif. Intell. 14 (2002) 189–216.
[29] Jianmin Ma, Hu. Lingling, Yu.hua. Qian, Object-oriented interval-set concept lattices, Int. J. Approx. Reason. 110 (2019) 64–81.
[30] Jesús Medina, Manuel Ojeda-Aciego, Jorge Ruiz-Calviòo, Formal concept analysis via multi-adjoint concept lattices, Fuzzy Sets Syst. 160 (2) (January 2009) 130–144.
[31] Jan Outrata, Vilem Vychodil, Fast algorithm for computing fixpoints of Galois connections induced by object-attribute relational data, Inf. Sci. 185 (1) (2012) 114–127.
[32] Nicolas Pasquier, Yves Bastide, Rafik Taouil, Lotfi Lakhal, Discovering frequent closed itemsets for association rules, in: International Conference on Database Theory, Springer, 1999, pp. 398–416.
[33] Ting Qian, Ling Wei, Jian-Jun Qi, Constructing three-way concept lattices based on apposition and subposition of formal contexts, Knowl. Based Syst. 116 (2017) 39–48.
[34] Yiyu Yao, Three-way granular computing, rough sets, and formal concept analysis, Int. J. Approx. Reason. 116 (2020) 106–125.
[35] Matei Zaharia, Tathagata Das, Haoyuan Li, Scott Shenker, Ion Stoica, Discretized streams: An efficient and fault-tolerant model for stream processing on large clusters, 4th USENIX Workshop on Hot Topics in Cloud Computing, 12–13, HotCloud'12, Boston, MA, USA, June, 2012, p. 2012.
[36] Mohammed Javeed Zaki and Mitsunori Ogihara, Theoretical foundations of association rules, in: 3rd ACM SIGMOD workshop on research issues in data mining and knowledge discovery, 1998, pp. 71–78.
[37] Huilai Zhi, Jianjun Qi, Ting Qian, Ling Wei, Three-way dual concept analysis, Int. J. Approximate Reasoning 114 (2019) 151–165.
[38] Huilai Zhi, Jianjun Qi, Ting Qian, Ling Wei, Three-way dual concept analysis, Int. J. Approx. Reason. 114 (2019) 151–165.
[39] Li Zou, Kuo Pang, Xiaoying Song, Ning Kang, Xin Liu, A knowledge reduction approach for linguistic concept formal context, Inf. Sci. 524 (2020) 165–183.